High-Level Design (HLD) Document

Project Title: *Shipment Pricing Prediction*
Domain: *Supply Chain Analytics*
Difficulty Level: *Intermediate*
Cloud Provider: *AWS*

Table of Contents

Abstract

This High-Level Design document provides a complete overview of the Shipment Pricing Prediction system. It covers the system's goals, design principles, technology stack, major components, core workflows, and operational overview. Designed to align stakeholders and developers, it sets the foundation for implementing a scalable, automated solution that predicts shipping costs with accuracy, consistency, and reliability using cloud infrastructure and MLOps practices.

The solution aims to support supply chain teams in predicting accurate shipment costs for various global routes and product types, improving procurement efficiency and reducing pricing errors.

---

1. Introduction

1.1. Why This Document Exists

This HLD exists to ensure that everyone involved—developers, project managers, stakeholders—shares a clear and common understanding of how the system is structured. Before diving into code or infrastructure setup, this document maps out the design vision, component responsibilities, and the flow of work.

1.2. Scope

This document defines the high-level architecture of the solution—from data handling and model training to deployment and automation. While it avoids programming specifics, it precisely outlines how the system will function and interact to achieve its goals.

1.3. Definitions

- **MLOps:** A set of practices for integrating ML model development, deployment, and maintenance in production.
- **CI/CD:** Continuous Integration/Continuous Deployment—the process that automates testing and deployment.
- **Artifact:** Output files generated during pipeline steps, such as trained model files or transformation objects.

---

2. General Description

2.1. Product Perspective

The system is a standalone web-based application that predicts shipping costs using a trained machine learning model. Users interact through a simple web interface, while behind the scenes the system ingests data, performs model operations, and provides predictions automatically.

2.2. Problem Statement

Calculating shipping prices manually takes a lot of time and often leads to mistakes. It becomes harder to manage as the number of shipments grows. The shipping cost can change depending on factors like weight, delivery location, vendor terms, and whether it's sent by air or sea.

Doing this manually causes delays and can lead to wrong decisions during procurement. This project solves that problem by using machine learning to predict shipping prices more accurately and faster. The system learns from past purchase order (PO) data and helps estimate the cost for each shipment line item.

2.3. Proposed Solution

We propose a cloud-based system built around a trained machine learning model. The solution uses Python for modeling, automates pipelines with MLOps tools, packages everything in containers via Docker, and deploys using AWS components. The system also includes tooling for tracking experiments, versioning datasets and models, and automatically redeploying updated models.

The goal of the **Shipment Pricing Prediction** project is to create a system that can **predict the cost of shipping** based on details such as shipment type, product category, weight, and location.

The system combines **machine learning**, **data analysis**, and **MLOps practices** to:

- Give **accurate pricing** to avoid overcharging or undercharging.
- Save **time and effort** by automating predictions.
- Provide **insights** on what factors affect shipment costs.
- Work well even with **large amounts of data** using cloud services.
- Keep results **consistent and reproducible** using version control and automation.

2.4. Further Improvements (Future Enhancements)

Possible Future Enhancements
In the future, we can make this system even smarter by adding:

- **Real-time shipment tracking** – So users can see exactly where their shipments are at any moment, reducing delays and improving trust.
- **Dynamic pricing** – Prices can automatically adjust based on demand, routes, or seasons, helping to increase profits and stay competitive.
- **Model performance monitoring with drift alerts** – The system can watch the model's accuracy over time and warn us if it starts making mistakes, so we can fix it before it affects customers.

These upgrades would make the platform **faster, smarter, and more reliable**, giving it a big advantage in the market.

2.5. Technical Requirements

To support robust performance and automation, the solution requires:

- **Development Stack:** Python, Pandas, NumPy, scikit-learn for ML; Matplotlib/Seaborn for EDA.
- **Deployment Stack:** FastAPI for serving predictions, Docker for consistent environments, AWS for hosting and storage (EC2, S3, ECR).
- **MLOps Tools:** DVC for dataset/model versioning, MLflow and Dagshub for experiment tracking, GitHub Actions for CI/CD automation.

2.6. Data Requirements

The system works with **structured shipment records** that include key details such as **weight, origin, destination, delivery time, and past shipping cost**. Data can be stored in **CSV files** or pulled directly from **MongoDB**.

To make sure the model works accurately, all incoming data must follow a **fixed schema**—with the same column names, formats, and data types—so that every record is clean, complete, and ready for processing. This consistency helps avoid errors during training and ensures the model's predictions are reliable.

2.7. Technology Stack

A curated list of the primary tools and why they were chosen:

Machine Learning & Data Processing

- **Python** – Main programming language.
- **Scikit-learn** – For model building and evaluation.
- **Pandas** – For data cleaning and manipulation.
- **NumPy** – For numerical calculations.

Data Analysis & Visualization *(Done in Jupyter Notebook during EDA)*

- **Matplotlib** – For basic plots.
- **Seaborn** – For advanced statistical visualizations.

Backend & API Development

- **FastAPI** – To create the web API and interface.

Cloud Infrastructure & Deployment

- **AWS S3** – Stores trained models and files.
- **AWS EC2** – Runs the FastAPI application.
- **AWS ECR** – Stores Docker images for deployment.

MLOps & Automation

- **DVC** – Tracks data and models.
- **MLflow** – Logs model experiments and metrics.
- **Dagshub** – Hosts MLflow remotely.
- **GitHub Actions** – Automates training and deployment.

Containerization

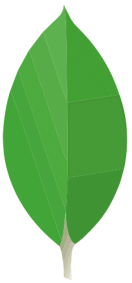
- **Docker** – Packages the app so it runs the same everywhere.


matplotlib


seaborn

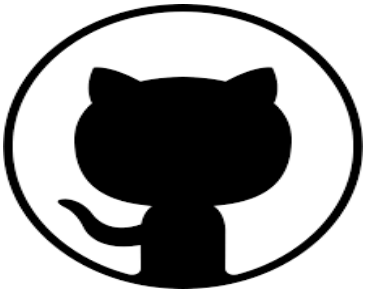

scikit
learn


jupyter




aws


git




GitHub Actions


DagsHub


mlflow


DVC


docker


FastAPI


HTML
CSS



NOTE : (No hardware-specific requirements are needed since everything runs in the cloud.)

2.8. Constraints

- Running the app in real time on AWS can increase costs.
- Loading large ML models may cause a slight delay in giving predictions.
- The system depends on AWS services being available and stable.

2.9. Assumptions

- The data we start with is clean and good quality.
- New training data will be available from time to time for model updates.
- AWS will run without major issues.
- If incoming data is very different from the training data, the system will detect it and handle it correctly.

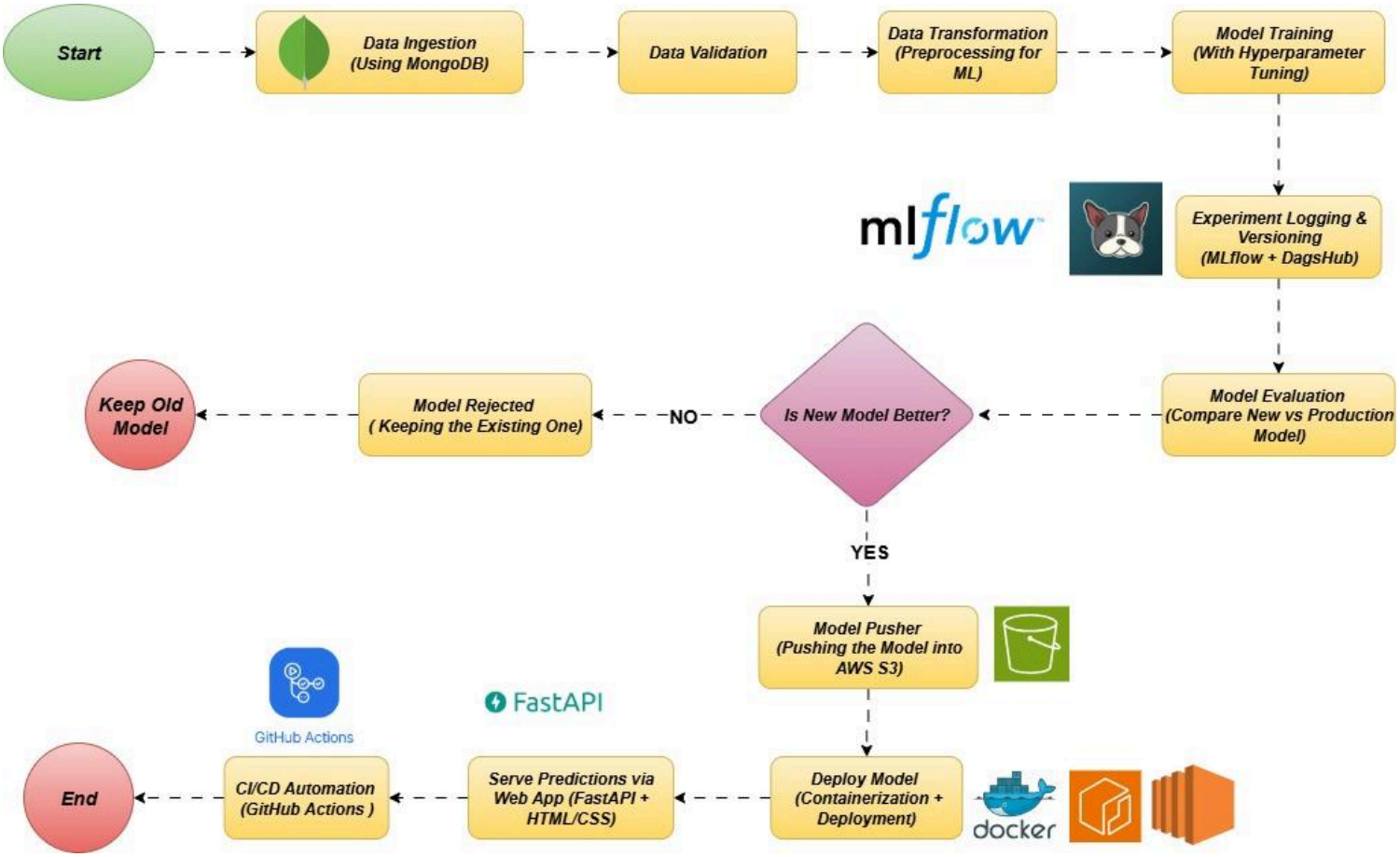
---

3. Design Details

3.1. Process Flow

The system follows a predictable, modular flow:

1. **Data ingestion** from CSV/MongoDB
2. **Validation** against schema and drift checks
3. **Transformation** of data for ML
4. **Model training** and hyperparameter tuning
5. **Evaluation** comparing new vs production model
6. **Deployment** if the new model performs better
7. **Serving predictions** via API
8. **Automation** through CI/CD pipelines



3.1.1. Model Training and Evaluation

The model is trained either by running a script manually or through our automated pipeline. All training details—like parameters used and results—are saved in MLflow for tracking. The model's performance is checked using RMSE, MAE, and R^2. The best-scoring model moves forward for deployment.

3.1.2. Deployment Process

Once we have the best model, it's packed into a Docker image and stored in AWS ECR. From there, it is deployed to an AWS EC2 instance. The FastAPI backend loads the model from S3 whenever a prediction is needed and sends results to the HTML/CSS web app instantly.

3.2. Event Logging

Our GitHub Actions pipeline records every step it takes—from training to deployment—so we can see exactly what happened and when. It also logs model performance and server health.

3.3. Error Handling

The system checks for problems at each stage—like wrong data formats or unusual patterns in new data. If a new model fails, the system will switch back to the last best model automatically. Clear error logs help us fix issues fast.

3.4. Performance

We keep track of how accurate the model is and how fast it responds to requests. We also measure how long training, packaging, and deployment take, so we know if the process is slowing down.

3.5. Reusability

The project is built in parts (data ingestion, validation, training, deployment, and prediction). This means we can change or upgrade one part without affecting the others.

3.6. Application Compatibility

The app works in any modern browser. The HTML/CSS frontend connects to FastAPI, which handles requests and predictions smoothly.

3.7. Resource Utilization

- **AWS EC2** → Runs the app and model
- **AWS S3** → Stores trained models
- **AWS ECR** → Stores Docker images
- **GitHub Actions** → Automates the entire process

This setup keeps costs reasonable and makes sure resources are used efficiently.

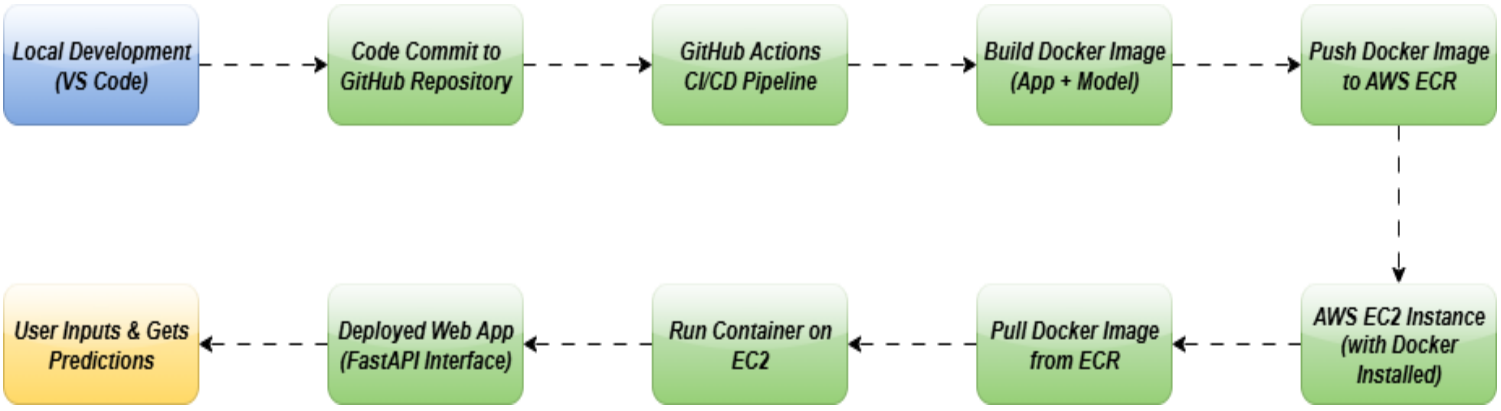


3.8. Deployment Architecture

When we update the code or data, the process runs automatically:

- A **code commit** starts the retraining process.
- The new model is **validated** to make sure it's better than the old one.
- If it passes, a **Docker container** is built with the model inside.
- The container is uploaded to **AWS ECR** and deployed to **AWS EC2**.

The **FastAPI server** picks up the new model and makes it live for predictions—usually within minutes.



4. Dashboards

4.1. KPIs (Key Performance Indicators)

- The system includes dashboards to show important information clearly and visually.
- These dashboards help both business and technical teams track performance.
- KPIs are used to measure how well the system and processes are working.
- Dashboards show summaries as well as allow users to explore detailed data.
- Data is updated regularly to reflect real-time or recent activity.
- The system can be easily updated to add new KPIs or connect more data sources.
- Dashboards are linked with data logs and pipelines to make sure the data is correct.
- Access control is used to ensure that each user sees only the information they are allowed to.
- The design is user-friendly, responsive, and works well on different devices.
- BI tools are used to create clean and easy-to-understand visuals like charts and graphs.

---

5. Conclusion

This High-Level Design (HLD) presents a clear and efficient system for predicting shipment prices. It uses cloud services, containerization, and MLOps practices to make the process **reliable, automated, and easy to maintain**. The design is built to handle growth, adapt to changes, and support new features in the future—ensuring accurate pricing predictions that can improve decision-making and reduce costs over time.
